

Below is a complete “starter” set of project documentation you can paste into a repo (for example as `README.md` + `/docs/\*`). It’s written for an **authorized** security scanning platform (only assets you own or have explicit permission to test).

Project Documentation: Web-Based Asset Scanner (Authorized)

1) Purpose and Summary

Build a web application that helps an organization **discover and assess its own attack surface** by scanning:

- * IPs, domains, subdomains, and subnets (in-scope only)
- * Open ports and services
- * Common misconfigurations and known vulnerabilities (non-exploit, detection-focused)
- * Reporting, prioritization, and remediation tracking

Core principle: Scans must be restricted to targets the user is authorized to test, with clear scope controls and abuse prevention.

2) Goals, Non-Goals, and Success Criteria

Goals

1. **Asset discovery**: map domains → subdomains → IPs → services.
2. **Port and service enumeration**: identify open ports, banners, TLS info.
3. **Vulnerability detection**: detect common CVEs/misconfigs using safe checks (no exploitation).
4. **Repeatable scans**: scheduled scans, diffing, trend reporting.
5. **Actionable output**: severity, evidence, affected asset, remediation notes.

Non-Goals (v1)

- * Exploitation frameworks (no payload delivery, no privilege escalation).
- * Scanning arbitrary internet targets without proof of authorization.
- * “Mass scanning” the public internet.
- * DDoS-style stress tests.

Success Criteria

- * Users can verify ownership/authorization and create a scoped project.
- * A scan run produces an asset inventory + findings in a structured report.
- * Findings support dedupe, severity ranking, and export (PDF/JSON/CSV).

- * Safe operations: rate limits, audit logs, scoped credentials, secure storage.

3) Personas and User Stories

Personas

- * **Security Engineer**: runs recurring scans, wants diffs/trends.
- * **Sysadmin/IT**: wants quick visibility and remediation steps.
- * **Manager**: wants risk summaries and compliance-friendly reporting.

User Stories (sample)

- * As a user, I can **create a project** with allowed targets and scanning limits.
- * As a user, I can **prove I control a domain** (DNS TXT or HTTP token).
- * As a user, I can **scan a subnet** with configurable rate limits and exclusions.
- * As a user, I can **see new findings since last scan** and track closure.
- * As a user, I can **export a report** for audit/compliance.

4) Functional Requirements

4.1 Project and Scope Management

- * Create project: name, environment (prod/stage/dev), tags.
- * Define scope:
 - * Domains (example.com)
 - * Subdomains (optional explicit list)
 - * IPs and CIDRs (10.0.0.0/24)
 - * Exclusions (do-not-scan list, sensitive hosts)
- * Ownership/authorization verification (at least one):
 - * DNS TXT record challenge
 - * HTTP well-known file challenge
 - * IP reverse token challenge (for internal)
 - * Manual approval workflow for teams

4.2 Discovery (Recon)

- * Passive discovery (preferred first):
 - * DNS enumeration, certificate transparency lookups (optional), existing inventory import
- * Active discovery (in-scope only):

- * Brute-force subdomain enumeration (wordlists under rate limits)
- * Resolve A/AAAA/CNAME, track historical resolution (optional)

4.3 Port Scanning and Service Fingerprinting

- * Scan TCP ports (configurable top ports, or custom list)
- * Optional UDP (v2+)
- * Service detection:
 - * Banner grabs (safe)
 - * TLS cert details (issuer, SANs, expiry)
 - * HTTP metadata (status codes, headers, title)

4.4 Vulnerability and Misconfiguration Checks (Safe Detection)

- * Checks should be ****non-destructive**** and avoid exploitation.
- * Examples of safe checks:
 - * Missing security headers
 - * Weak TLS configuration indicators
 - * Known vulnerable version detection by banner + fingerprint (with confidence)
 - * Publicly exposed admin panels (informational)
- * Each finding must include:
 - * Asset reference (domain/IP/port/service)
 - * Severity
 - * Evidence (request/response metadata, banner excerpt)
 - * Confidence level (high/medium/low)
 - * Recommended remediation

4.5 Reporting and Workflow

- * Dashboard:
 - * Asset inventory
 - * Findings by severity and category
 - * Trends over time
- * Findings:
 - * Dedupe logic (same signature, same asset)
 - * State: open, accepted risk, fixed, false positive
 - * Comments and assignments (optional v2)
- * Exports:
 - * JSON (for integration)

- * CSV
- * PDF report (executive + technical appendices)

4.6 Scheduling and Notifications

- * Scheduled scans (daily/weekly/monthly)
- * Alerting when:
 - * New critical finding
 - * New exposed port/service
 - * Certificate expiry approaching

5) Non-Functional Requirements

Security

- * Strong auth (OIDC/SAML optional later)
- * Role-based access control: Owner, Admin, Analyst, Viewer
- * Audit logging: who ran scans, changed scope, exported reports
- * Secrets storage: encrypted at rest (use KMS if available)

Performance and Reliability

- * Async job execution (queue + workers)
- * Timeouts for each scan stage
- * Retries with backoff for transient failures
- * Rate limiting per project and per target

Compliance and Abuse Prevention

- * Proof-of-ownership gates for internet-facing domains
- * Explicit scope acceptance before scans start
- * Hard caps: max hosts per scan, max requests/sec, max concurrency
- * “Kill switch” to stop scans quickly

6) High-Level Architecture

Components

1. **Web UI** (Frontend)

- * Project setup, scope, scan runs, findings, reports

2. ****API Server**** (Backend)

- * Auth, projects, scope validation, scan orchestration, findings lifecycle

3. ****Job Queue****

- * Enqueue scan tasks, schedule recurring runs

4. ****Worker Fleet****

- * Executes scanning pipeline stages in isolated containers

5. ****Database****

- * Projects, targets, scan runs, assets, findings, evidence

6. ****Object Storage****

- * Raw artifacts: scan logs, partial outputs, report files

Recommended Reference Stack (example)

- * Frontend: Next.js (React) or similar

- * Backend: FastAPI / Django / Node

- * Queue: Redis + Celery (or BullMQ), or RabbitMQ

- * DB: PostgreSQL

- * Workers: Docker containers, run as restricted user

- * Deployment: Docker Compose (dev), Kubernetes (prod)

7) Scanning Pipeline Design

Stage Order (safe default)

1. ****Scope validation****

- * Expand CIDRs, resolve domains, apply exclusions

2. ****Passive recon****

- * DNS records, known subdomains (from user/import), cert metadata (optional)

3. ****Active discovery****

- * Subdomain enumeration (rate-limited)

4. ****Port scan****

- * Configurable port sets and timing templates

5. ****Service fingerprint****

- * HTTP/TLS/banner metadata

6. **Vuln/misconfig checks**

- * Safe checks only, strict timeouts

7. **Normalize + dedupe**

- * Convert to common asset model

8. **Persist + report**

- * Create findings, compute diffs, generate exports

Isolation and Safety

- * Workers run in containers with:

- * No privileged mode

- * Read-only filesystem where possible

- * Limited outbound access if feasible (egress policy)

- * Command execution must be hardened (no untrusted concatenation).

8) Data Model (Proposed)

Core Tables

- * **organizations**

- * **users**

- * **projects**

- * **project_members**

- * **scopes**

- * allowed_domains, allowed_cidrs, allowed_ips, exclusions

- * **targets**

- * normalized entities: domain, subdomain, ip, cidr

- * **scan_runs**

- * status, start/end, initiated_by, configuration snapshot

- * **assets**

- * ip, hostname, port, service, protocol, metadata

- * **findings**

- * finding_type, severity, confidence, status, first_seen, last_seen

- * **evidence**

- * linked to finding: banner excerpt, http headers, timestamps

* **artifacts**

- * raw logs/output files pointers

Finding Key (for dedupe)

A stable hash built from:

- * finding_type + normalized asset identifier + port + protocol + signature version

9) API Surface (Example)

Projects

- * `POST /api/projects`
- * `GET /api/projects`
- * `GET /api/projects/{id}`

Scope

- * `PUT /api/projects/{id}/scope`
- * `POST /api/projects/{id}/verify-domain` (returns challenge)
- * `POST /api/projects/{id}/verify-domain/confirm`

Scans

- * `POST /api/projects/{id}/scan-runs`
- * `GET /api/projects/{id}/scan-runs`
- * `GET /api/scan-runs/{scan\_id}`

Findings

- * `GET /api/projects/{id}/findings?severity=&status=`
- * `PATCH /api/findings/{finding\_id}` (status, notes)

Reports

- * `POST /api/scan-runs/{scan\_id}/export?format=pdf|json|csv`
- * `GET /api/exports/{export\_id}`

10) Tooling Strategy (How scanning is implemented)

Principle

Prefer ****well-tested scanners**** as pluggable stages, wrapped by your worker runtime, and normalize output into your data model.

Plugin Interface (concept)

Each stage is a plugin with:

- * Input: list of targets/assets + config
- * Output: JSON lines of discovered assets/findings
- * Constraints: timeout, concurrency, rate limit

Output Normalization

Convert tool-specific data into:

- * Asset: `ip, hostname, port, protocol, service, metadata`
- * Finding: `type, severity, confidence, evidence`

11) Security Design Notes (Web App)

Key Risks

- * ****Abuse****: users scanning targets they do not own
- * ****SSRF****: scanner or API can be used to reach internal services
- * ****Command injection****: unsafe invocation of CLI scanners
- * ****Data leakage****: storing sensitive banners/headers/credentials
- * ****Multi-tenancy****: cross-project access

Mitigations

- * Ownership verification gates and strict scope enforcement
- * Egress policies for workers (block RFC1918 if not allowed, or require explicit enable)
- * Use parameterized execution, no shell concatenation
- * Encrypt sensitive data at rest; redact secrets from evidence
- * RBAC + project-level authorization checks everywhere
- * Audit logs for scope changes and scan runs

12) Testing Plan

Unit Tests

- * Scope parsing and normalization (CIDR expansion, exclusions)
- * Dedupe hashing logic
- * Severity mapping and confidence rules

Integration Tests

- * Full pipeline against a local test lab (Docker-based vulnerable-by-design services)
- * Report generation outputs valid PDF/CSV/JSON

Security Tests

- * AuthZ tests for multi-tenant separation
- * Input fuzzing for scope and scan configuration
- * Validate workers cannot execute arbitrary commands via config

13) Deployment and Operations

Environments

- * **Dev**: single-machine Docker Compose
- * **Staging**: same as prod but smaller
- * **Prod**: Kubernetes or managed containers

Observability

- * Structured logs (scan_id, project_id, stage)
- * Metrics: scans/hour, avg duration, errors/stage, queue depth
- * Tracing optional

Backups

- * DB backups daily
- * Artifact retention policy (configurable), with secure deletion

14) Roadmap (Practical Milestones)

Milestone 1: MVP (Inventory + Port Scan)

- * Projects + scope
- * Domain verification

- * Scan run orchestration
- * Port scan + service metadata
- * Basic UI + exports (JSON/CSV)

Milestone 2: Subdomain Discovery + Diffing

- * Subdomain enum + DNS resolution
- * Asset graph (domain → subdomain → IP)
- * Compare scans (new/removed ports, hosts)

Milestone 3: Safe Vulnerability Checks + Reporting

- * Misconfiguration checks (HTTP headers, TLS hygiene)
- * Version-based detection with confidence
- * PDF executive report

Milestone 4: Scheduling + Notifications

- * Cron-like schedules
- * Alerts for critical changes

15) Repo Structure (Suggested)

```

/frontend
/backend
/workers
  /plugins
/docs
  PRD.md
  ARCHITECTURE.md
  THREAT_MODEL.md
  API.md
  DATA_MODEL.md
/docker
docker-compose.yml
README.md

```

Notes on Responsible Use

This kind of platform should be built and operated with strong guardrails: explicit authorization, scope limits, rate limits, and audit logs. That's not just "legal boilerplate", it directly affects whether the product can be safely offered to real users.

If you want, share what stack you prefer (Python vs Node, single VPS vs Kubernetes). I can convert the above into a repo-ready set of files (^/docs/*.md`) and a concrete MVP backlog (epics → tickets) with acceptance criteria.